

ラボ 3.3: 手動暗号化

1. はじめに

1.1. 必要なリソース

- ControlThings プラットフォーム VM

1.2. 概要

このラボでは、英国政府通信本部（GCHQ）のオープンソースツールであるCyberChefを使用して暗号操作を実行します

1.3. 目的

- ハッシュ化を実行する
- 暗号化を実行する
- 復号化を実行する
- ペイロード署名を実行する

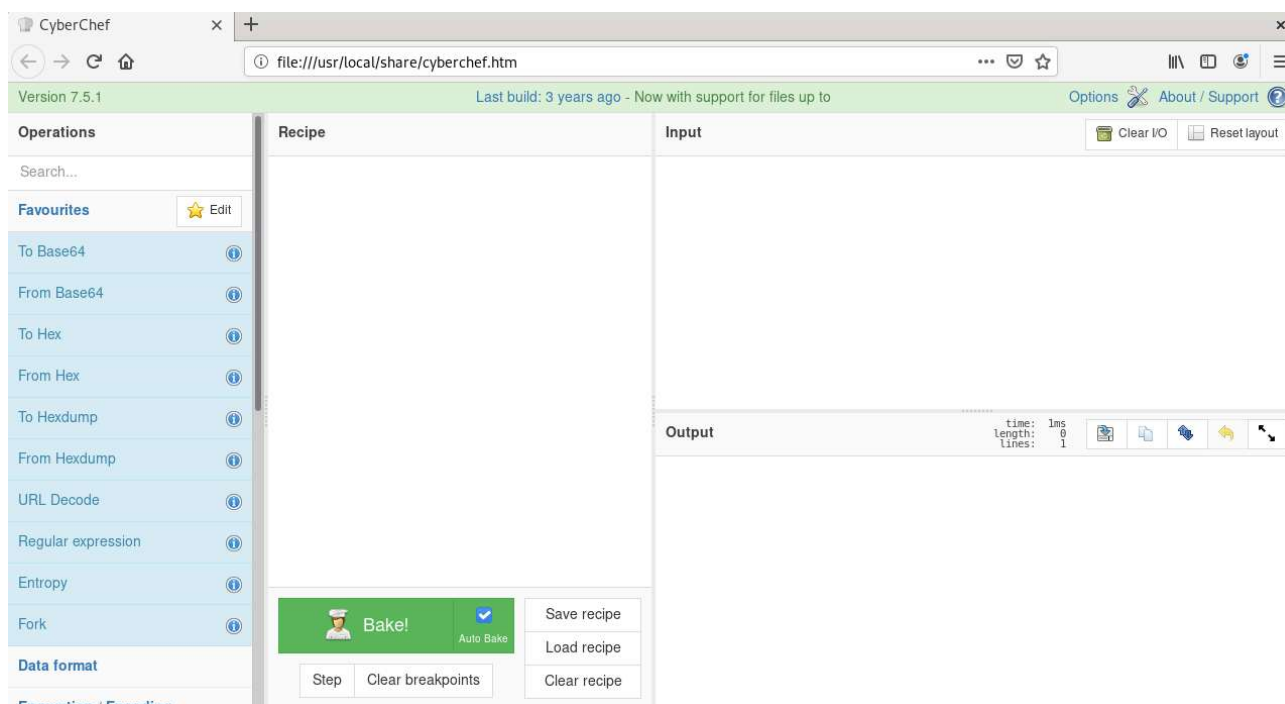
2. OpenSSLとCyberChef

暗号操作を実行するためのツールやプログラミングライブラリは数多くあります。このラボでは、OpenSSLとCyberChefという2つの異なるツールを使用します。1つのツールで操作を実行し、その操作を2つ目のツールで確認します

CyberChefは、Webブラウザで手動で暗号化操作を行うためのWebアプリケーションです。これは英国政府通信本部（GCHQ）によって開発され、[GitHub](#)でオープンソースとして公開されています。今回の演習では、ControlThings Platformに（様々な理由により）古いバージョンのCyberChefがローカルにインストールされています。ただし、必要に応じて、<https://gchq.github.io/CyberChef>にアクセスすることで、どのコンピュータからでも最新バージョンにアクセスできます。すべての操作はWebブラウザのJavaScriptを介して実行されるため、入力された情報はバックエンドのWebサーバーに渡されることはありません。

Activities メニューでCyberChefを検索して起動します

✓ 成功



注意

ControlThingsプラットフォームへのCyberChefのインストールで問題が発生した場合は、いつでも上記のリンク先の公開バージョンにフォールバックできます

OpenSSLは、SSL（現在は非推奨）およびTLS向けの世界で最も広く利用されている暗号ライブラリであり、市場シェアは約66%と推定されています。HTTPSを使用するWebサーバーから、TLSトンネルと統合されたSCADAプロトコルを使用する組み込みICSデバイスまで、事実上あらゆる業界の様々なプロトコルで使用されています。OpenSSLは、手動操作用のコマンドラインインターフェースを備えた汎用暗号ライブラリでもあります。100%オープンソースでありながら、商用グレードの堅牢性を備えて構築および保守されています。OpenSSLの詳細については、[OpenSSLのウェブサイト](#)をご覧ください。

後ほどOpenSSLコマンドで使用する新しいコマンドラインターミナルを開きます。

```
echo -n "I am taking ICS410" > plaintext
```

-n テキストの末尾に改行を追加しないように echo に指示する を忘れないようにしてください。

「I am taking ICS410」というASCII文字列を含む新しいファイルを作成しました `plaintext`。このファイルは、すべての暗号化操作に使用します。

```
cat plaintext
```

✓ 成功

```
I am taking ICS410%
```

はい、これは奇妙に見えるはずです。テキストの末尾に改行が含まれていないため、`cat`は改行を追加しません。その結果、ターミナルシェルであるZshは%末尾に改行を追加し、出力に部分的な行があり、コマンドの実行が完了する前に改行文字を受け取らなかったことを示します

Linuxのコマンドプロンプトからテキストファイルを読み取る場合、通常はこの方法で行います。ただし、この方法はASCII、UTF-8、またはUTF-16でエンコードされたファイルにのみ適用されます。UTF-8はUnicodeエンコード方式で、英語以外の言語に必要な特殊文字に対応しています。UTF-8は最も一般的で、現代のコンピュータシステムのほとんどでデフォルトで使用されています。他のエンコードのファイルや、生のバイナリデータを含むファイルを読み取りたい場合は、のような生の形式でデータを表示できるツールを使用する必要があります `xxd`。

```
xxd plaintext
```

✓ 成功

```
00000000: 4920 616d 2074 616b 696e 6720 4943 5334  I am taking ICS4
00000010: 3130                                     10
```

出力がこれとまったく同じでない場合は、戻ってファイルを再作成してください。そうしないと、このラボの残りの部分では何も正しく動作しなくなります。

前のラボで説明したように、`xxd` 出力には3つの列があります。

1. ファイル内の場所のアドレス
2. ファイルの生の16進ダンプ
3. ファイルのASCIIデコード

生のバイナリファイルを見ている場合、ASCII列は一見ランダムな文字で読み取れません。その場合、多くの、文字も表示されます。これらの文字は、時にはリテラル文字を意味しますが、ほとんどの場合、印刷できないASCII文字、またはASCIIに割り当てられていない16進値を表します。ASCIIでは、256通りの16進値のうち128通りしか使用されません。

? 前回は、出力をに `xxd` パイプ (`()`) しました。今回はなぜダメなのでしょう? | `less`

前回のラボでは大きなファイルを使用していたため、出力をパイプで接続しただけです `less`。今回は小さなファイルを使用しているため、すべての出力が常に画面に収まるはずです。

3. ハッシュ

OpenSSLでは、多くのハッシュ関数から選択できます。OpenSSLがサポートしている関数のリストは次のとおりです

```
openssl dgst -list
```

✓ 成功

Supported digests:

<code>-blake2b512</code>	<code>-blake2s256</code>	<code>-md4</code>
<code>-md5</code>	<code>-md5-sha1</code>	<code>-ripemd</code>
<code>-ripemd160</code>	<code>-rmd160</code>	<code>-sha1</code>
<code>-sha224</code>	<code>-sha256</code>	<code>-sha3-224</code>
<code>-sha3-256</code>	<code>-sha3-384</code>	<code>-sha3-512</code>
<code>-sha384</code>	<code>-sha512</code>	<code>-sha512-224</code>
<code>-sha512-256</code>	<code>-shake128</code>	<code>-shake256</code>
<code>-sm3</code>	<code>-ssl3-md5</code>	<code>-ssl3-sha1</code>
<code>-whirlpool</code>		

現在開発されているすべての最新ソフトウェアは、一般的に SHA3 ファミリを使用しているはずなので、これに重点を置きます。

```
openssl dgst -sha3-512 plaintext
```

✓ 成功

SHA3-512(plaintext)=

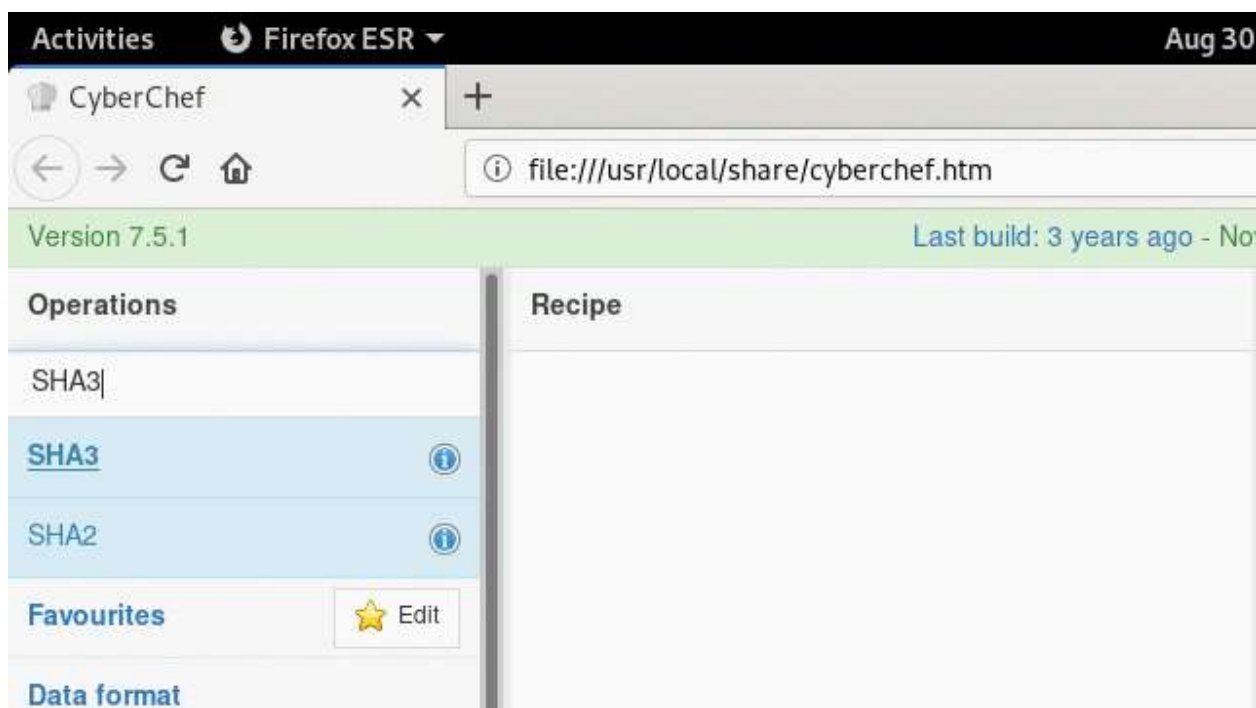
308d7ad70b890891ec250b728e050be154ba58b842ab7d0758f88a8cb4454d3366e934785272c0e55397ee9e81cff5bcf3186c1f51.

元の18バイトのプレーンテキストファイルと比べて、どれだけ長いかわかりでしょう。実は、512ビットのハッシュを使用しており、これは64バイト（512ビット/バイトあたり8ビット）の長さです。

CyberChef が同じハッシュを生成するかどうか確認してみましょう。

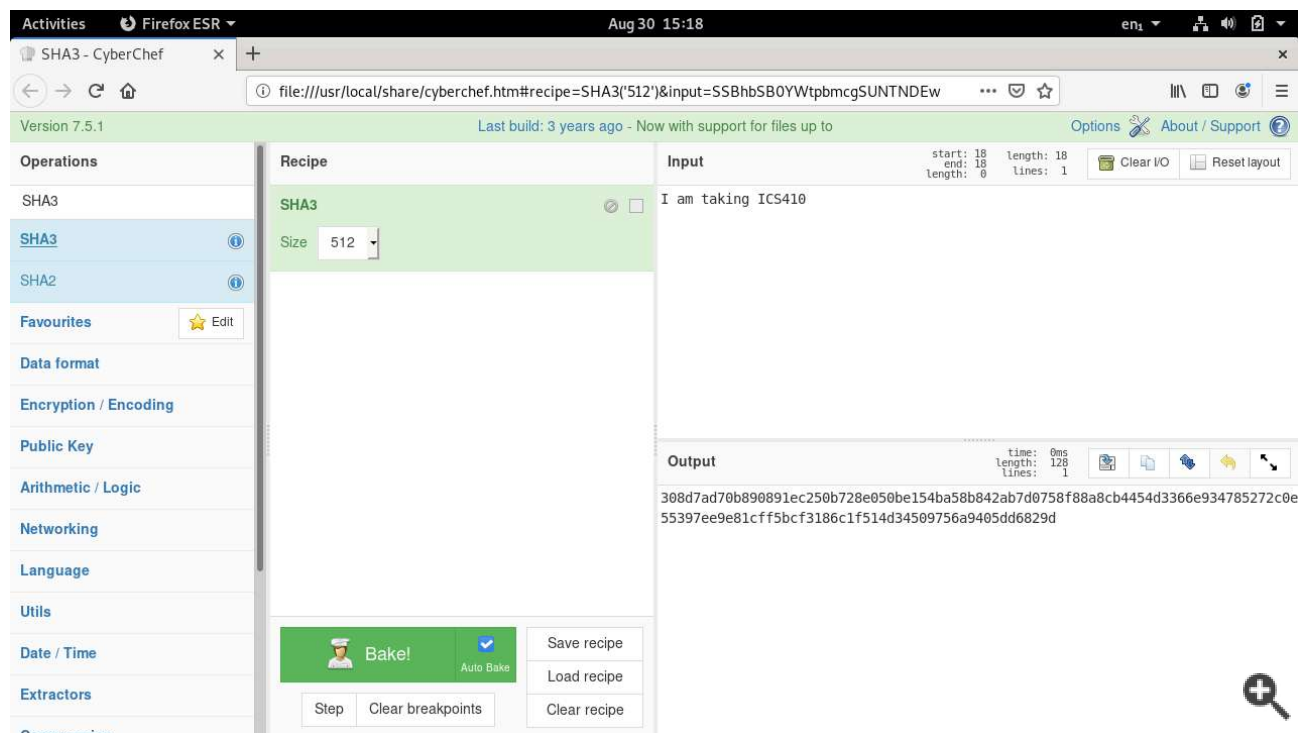
CyberChefでは、の下に **Operations** 入力して **SHA3** 正しい関数を検索し、関数をセクション **SHA3** にドラッグします。Recipe

✓ 成功



セクションに Input、前と同じメッセージを入力します I am taking ICS410

✓ 成功



入力内容を正しく入力し、Enter キーを押さない限り、Output セクションは OpenSSL と同じになるはずで

では、コマンドプロンプトとOpenSSLに戻り、もう少しテストしてみましょう。別のファイル名でファイルのコピーを作成し、ハッシュが変化するかどうかを確認してみましょう。

```
cp plaintext plaintext1
```

```
openssl dgst -sha3-512 plaintext1
```

✓ 成功

```
SHA3-512(plaintext1)=
308d7ad70b890891ec250b728e050be154ba58b842ab7d0758f88a8cb4454d3366e934785272c0e55397ee9e81cff5bcf3186c1f514d34509756a9405dd6829d
```

ハッシュが変更されていないことに注目してください。ハッシュはファイルの内容のものであり、ファイル名は含まれていません

ここで、1つの文字を小文字から **a** 大文字に変更した 2 番目のファイルを作成しましょう **A**。

```
echo -n "I am tAking ICS410" > plaintext2
```

```
openssl dgst -sha3-512 plaintext2
```

✓ 成功

```
SHA3-512(plaintext2)=  
8057adfd89868a42a23b749c28d677806a86176716c405a21cc4644d871cbf1f44de45efa0a31d6196b12041caf702d0d484740ee21
```

たった 1 文字の変更で、完全に異なるハッシュが生成されたことに注意してください。

講義で述べたように、ハッシュを使用することでファイルの内容が変更されたかどうかを判断できますが、メッセージを変更したい悪意のある攻撃者がいる場合、ハッシュにアクセスできればハッシュも変更することができます。中間者攻撃 (MitM) はまさにその好例です。メッセージとハッシュは通常同じプロトコルで転送されるため、他の暗号保護手段が使用されない限り、攻撃者は両方を変更できます。

4. メッセージ認証コード

メッセージ認証コード (MAC) は、ネットワークトラフィックを中間者攻撃から保護する方法の一つです。これは、ハッシュを共有鍵で暗号化することで保護すると考えてください。アルゴリズムで実際に行われる処理と厳密には一致しませんが、プロセスを理解する上で参考になります。これはペイロード署名の基本的な形式とも考えられており、特に非対称暗号化に対応できない低リソースの組み込みデバイスを使用する場合に有効です。

```
openssl dgst -sha3-512 -hmac "letmein" plaintext
```

✓ 成功

```
HMAC-SHA3-512(plaintext)=  
d005876299ba252aa3b4777e8bc88a7b0545115b451ac4f31244abc6b03cfff3908164d656a67d43ed5151f14215d62b69cd12c450b1
```

HMACH 残念ながら、CyberChefはの使用をサポートしていません SHA3-512 が、SHA-512 SHA2ファミリーの512ビット版である はサポートしています。そのため、その前にまずOpenSSLでHMACを計算しましょう。

```
openssl dgst -sha512 -hmac "letmein" plaintext
```

✓ 成功

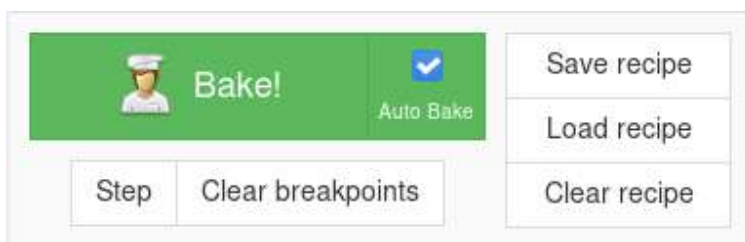
```
HMAC-SHA512(plaintext)=  
a71b9190fbc041365494f536d0cef6c7d4a4d4a68f0bc91ccd822d9057e6810419a73867b442f3a379b11bff7eac10187a40f5d26a;
```

出力の長さは 512 ビットで同じですが、以前使用していたより新しく強力な SHA3 ファミリではなく、SHA2 ファミリを使用しているため、出力が異なることに注意してください。

CyberChef でも同じ出力が得られるか確認してみましょう。

CyberChefウィンドウの下部にある `Clear recipe` ボタンを押します

≡ 例



関数を検索し、セクション HMAC にドラッグします Recipe

関数内でパスワードを HMAC 入力し、ハッシュ関数を次のように変更します。 letmein SHA512

Input セクションにまだ含まれていることを確認してください I am taking ICS410

✓ 成功

The screenshot shows the CyberChef web application in a Firefox ESR browser window. The address bar shows the file path: `file:///usr/local/share/cyberchef.htm#recipe=HMAC('letmein','SHA512')&input=SSBhbSB0YWtpbm`. The interface is divided into several sections:

- Operations:** A sidebar on the left with a list of operations including 'mac', 'Format MAC addresses', 'Extract MAC addresses', 'HMAC', 'Disassemble x86', 'Generate HOTP', 'Favourites', 'Data format', 'Encryption / Encoding', 'Public Key', 'Arithmetic / Logic', 'Networking', 'Language', and 'File'.
- Recipe:** The central area shows an 'HMAC' recipe selected. The 'Password' field contains 'letmein' and the 'Hashing function' is set to 'SHA512'.
- Input:** The input field contains the text 'I am taking ICS410'.
- Output:** The output field displays a long hexadecimal hash: `a71b9190fbc041365494f536d0cef6c7d4a4d4a68f0bc91ccd822d9057e6810419a73867b442f3a379b11bfff7eac10187a40f5d26aae66660ec0162a8135da50`.
- Buttons:** At the bottom, there are buttons for 'Bake!', 'Auto Bake', 'Save recipe', 'Load recipe', 'Clear recipe', 'Step', and 'Clear breakpoints'.

CyberChef も同じ HMAC を生成するために登場したようです。

5. 対称暗号化

ハッシュ関数とMAC関数は、メッセージが変更されたかどうかを検出するために使用できるダイジェストを生成しますが、メッセージ自体を保護するものではありません。メッセージを保護したい場合は、暗号化を使用します

OpenSSL がサポートする対称暗号化関数は次のとおりです。

```
openssl enc -list
```

✓ 成功

Supported ciphers:

-aes-128-cbc	-aes-128-cfb	-aes-128-cfb1
-aes-128-cfb8	-aes-128-ctr	-aes-128-ecb
-aes-128-ofb	-aes-192-cbc	-aes-192-cfb
-aes-192-cfb1	-aes-192-cfb8	-aes-192-ctr
-aes-192-ecb	-aes-192-ofb	-aes-256-cbc
-aes-256-cfb	-aes-256-cfb1	-aes-256-cfb8
-aes-256-ctr	-aes-256-ecb	-aes-256-ofb
-aes128	-aes128-wrap	-aes192
-aes192-wrap	-aes256	-aes256-wrap
-aria-128-cbc	-aria-128-cfb	-aria-128-cfb1
-aria-128-cfb8	-aria-128-ctr	-aria-128-ecb
-aria-128-ofb	-aria-192-cbc	-aria-192-cfb
-aria-192-cfb1	-aria-192-cfb8	-aria-192-ctr
-aria40a-192-ecb	-aria-192-ofb	-aria-256-cbc
-aria-256-cfb	-aria-256-cfb1	-aria-256-cfb8
-aria-256-ctr	-aria-256-ecb	-aria-256-ofb
-aria128	-aria192	-aria256
-bf	-bf-cbc	-bf-cfb
-bf-ecb	-bf-ofb	-blowfish
-camellia-128-cbc	-camellia-128-cfb	-camellia-128-cfb1
-camellia-128-cfb8	-camellia-128-ctr	-camellia-128-ecb
-camellia-128-ofb	-camellia-192-cbc	-camellia-192-cfb
-camellia-192-cfb1	-camellia-192-cfb8	-camellia-192-ctr
-camellia-192-ecb	-camellia-192-ofb	-camellia-256-cbc
-camellia-256-cfb	-camellia-256-cfb1	-camellia-256-cfb8
-camellia-256-ctr	-camellia-256-ecb	-camellia-256-ofb
-camellia128	-camellia192	-camellia256
-cast	-cast-cbc	-cast5-cbc
-cast5-cfb	-cast5-ecb	-cast5-ofb
-chacha20	-des	-des-cbc
-des-cfb	-des-cfb1	-des-cfb8
-des-ecb	-des-edc	-des-edc-cbc
-des-edc-cfb	-des-edc-ecb	-des-edc-ofb
-des-edc3	-des-edc3-cbc	-des-edc3-cfb
-des-edc3-cfb1	-des-edc3-cfb8	-des-edc3-ecb
-des-edc3-ofb	-des-ofb	-des3
-des3-wrap	-desx	-desx-cbc
-id-aes128-wrap	-id-aes128-wrap-pad	-id-aes192-wrap
-id-aes192-wrap-pad	-id-aes256-wrap	-id-aes256-wrap-pad
-id-smime-alg-CMS3DESwrap	-rc2	-rc2-128
-rc2-40	-rc2-40-cbc	-rc2-64
-rc2-64-cbc	-rc2-cbc	-rc2-cfb
-rc2-ecb	-rc2-ofb	-rc4
-rc4-40	-seed	-seed-cbc
-seed-cfb	-seed-ecb	-seed-ofb
-sm4	-sm4-cbc	-sm4-cfb
-sm4-ctr	-sm4-ecb	-sm4-ofb

ハッシュの例と同様に、より強力な対称暗号化関数の一つである[aes-256-cbc](#)を使用します。これは、256ビット鍵のAESアルゴリズムで、暗号ブロック連鎖（CBC）モードを使用して、1ブロック（AESの場合は128ビット、16バイト）を超えるメッセージの処理方法を決定します。

暗号化を正しく行うには、他にも多くの入力が必要です。以下のコマンドをコピー＆ペーストすることをお勧めします。

```
openssl enc -aes-256-cbc -pbkdf2 -md sha512 -iter 10000 -S 0 -iv 0 -in plaintext | xxd
```

パスワードの入力を求められた場合は、`letmein` 両方の回数使用してください。

? OpenSSL オプションとは何でしょうか?

オプション	説明
enc	暗号化サブコマンド
-aes-256-cbc	暗号化機能
-pbkdf2	パスワードをより安全にする鍵導出スキーム
-md sha512	PBKDF2のハッシュ関数にSHA512を使用します
-iter 10000	PBKDF2とSHA512を10,000回使用する
-S 0	nullソルト値を使用します。クラス外ではこのオプションを削除してください。
-iv 0	AESでヌル初期化ベクトル（IV）を使用する
-k letmein	パスワード。絶対にしないでください

✓ 成功

```
hex string is too short, padding with zero bytes to length
hex string is too short, padding with zero bytes to length
00000000: 5361 6c74 6564 5f5f 0000 0000 0000 0000  Salted__.....
00000010: 2921 d667 ccbc eb24 72e1 7d8c 17a4 a98a  )!.g...$r.}.....
00000020: bb94 a17a 5d81 8d4f 2c83 b495 f867 503e  ...z]..0,...gP>
```

OpenSSLの出力をパイプして、`xxd` 結果をすぐに確認できるようにしていることに注意してください

16進文字列が短すぎるというエラーメッセージが2つ表示されています。これは `salt` と `iv` オプションに関するもので、それぞれ16個のゼロが含まれている必要があります。ただし、エラーメッセージには「すべてのゼロが自動的に入力される」というメッセージも表示されています。これはまさに私たちが求めていたことであり、ゼロを入力する必要はありませんでした。

実際の出力では、ASCII領域にソルトが付加されていること `xxd` がわかります。これは、パスワードにソルトが付加されていることを示す分かりやすいヒントで、ソルトはASCII領域に付加されています。テキストの左側の16進数セクションを見ると、ピリオドは8つのヌルバイトを表しており、これは16進数で16個のゼロに相当します。 `Salted__..... 0000 0000 0000 0000`

出力の 2 行目と 3 行目は、関数 `xxd` から実際に出力された暗号文です `aes-256-cbc`。各行が 16 バイトを表す場合、暗号文は 32 バイトであることを意味します。AES-256 は 128 ビット (または 16 バイト) のブロック暗号であり、一度に 16 バイトのみを暗号化することを思い出してください。プレーンテキスト ファイルの長さは 18 バイトで、AES の 16 バイト ブロックよりも長いいため、プレーンテキスト メッセージの最初のブロックは `I am taking ICS4` に暗号化されています `2921 d667 ccbc eb24 72e1 7d8c 17a4 a98a`。プレーンテキスト メッセージの残りの部分は `10` 2 バイトの長さしかありませんが、それでも常に 16 バイトを生成する AES 暗号化プロセスを経る必要があります。したがって、はで `10` に暗号化されています。 `aes-256-cbc bb94 a17a 5d81 8d4f 2c83 b495 f867 503e`

それでは、CyberChefで復号してみましょう。パスワードの保護を強化するためにPBKDF2を使用しているため、2段階に分けて復号する必要があります。

CyberChefで `clear recipe` ボタンを押します

関数を検索し `Derive PBKDF2 key` でレシピにドラッグします

`passphrase` をに設定してください `letmein`。これは私たちが使用したパスワードです。

設定 `Key size` する `256`

これは AES-256 派生キー用です。128 は AES-128 用です。

設定 `Iterations` する `10000`

これは OpenSSL に伝えたものと一致しており、2020 年の最小値と考えられます。

設定 `Hashing function` する `SHA512`

もう一度、OpenSSL に伝えた内容と一致します。

設定 `Salt` する `0000 0000 0000 0000`

ここでは16個のゼロをすべて入力する必要があります。CyberChefでは不正行為は許可されませんが、余分なスペースは有効な16進文字ではないため無視されます。

ボタンをクリックし `Bake`、数秒待つ、`Output` セクションが現在の設定を反映していることを確認します。

10,000 回の反復が完了するまでに時間がかかるため、自動ベイク機能が正しく動作しないことがあります。

✓ 成功

The screenshot shows the CyberChef web application interface. The browser address bar displays the URL: `file:///usr/local/share/cyberchef.htm#recipe=Derive_PBKDF2_key({'option':'UTF8','string':'letmein'...`. The application version is 7.5.1, and it notes 'Last build: 3 years ago - Now with support for files up to'. The left sidebar contains a menu with categories like Operations, Favourites, Data format, Encryption / Encoding, Public Key, Arithmetic / Logic, Networking, Language, Utils, Date / Time, Extractors, and Compression. The 'Derive PBKDF2 key' recipe is selected and configured with the following settings: Passphrase (UTF8, letmein), Key size (256), Iterations (10000), Hashing function (SHA512), and Salt (Hex, 0000 0000 0000 0000). The 'Output' section shows the resulting key: `59b7220b92cc0304da830b5f79c9da39cee1b844127f6aa01dd449b97dc1ccb1`. The interface also includes buttons for 'Bake!', 'Auto Bake', 'Save recipe', 'Load recipe', 'Clear recipe', 'Step', and 'Clear breakpoints'.

CyberChefは導出したPBKDF2鍵を次のように計算しました。

59b7220b92cc0304da830b5f79c9da39cee1b844127f6aa01dd449b97dc1ccb1

これを取得したら、暗号文を復号化できます

CyberChefで Clear recipe ボタンを押します

関数を検索し AES Decrypt でレシピにドラッグします

Key 先ほど導出したPBKDF2キーを設定します

設定 IV する 0000 0000 0000 0000 0000 0000 0000 0000

セクションに Input 16進数の暗号文を貼り付けます

2921 d667 ccbc eb24 72e1 7d8c 17a4 a98a bb94 a17a 5d81 8d4f 2c83 b495 f867 503e

✓ 成功

The screenshot shows the CyberChef web application in a Firefox ESR browser window. The address bar shows the URL: `file:///usr/local/share/cyberchef.htm#recipe=AES_Decrypt({'option':'Hex','string':'59b7220b92cc03'}`. The interface displays a recipe named "AES Decrypt" with the following settings:

- Key:** Hex, 844127f6aa01dd449b97dc1ccb1
- IV:** Hex, 0000 0000 0000 0000
- Mode:** CBC
- Input:** Hex
- Output:** Raw
- GCM Tag:** Hex

The input field contains the hex string: `2921 d667 ccbc eb24 72e1 7d8c 17a4 a98a bb94 a17a 5d81 8d4f 2c83 b495 f867 503e`. The output field shows the result: `I am taking ICS410`. The interface also includes buttons for "Bake!", "Auto Bake", "Save recipe", "Load recipe", "Clear recipe", "Step", and "Clear breakpoints".

6. ボーナスチャレンジ

他の参加者のラボの完了を待っている場合、またはこのラボで紹介された概念をさらに深く探求したい場合は、以下のアイデアを試してみてください。これらのチャレンジにはチュートリアルは用意されていませんが、行き詰まった場合は、コース作成者のジャスティンに取り組みの過程を見せていただければ、喜んでヒントをくれるでしょう。justin [@controlthings.io](mailto:justin@controlthings.io)までメールでお問い合わせください

以下のOpenSSLコマンドを実行し、独自のパスワードを選択し、CyberChefを使用して結果を復号化する

```
openssl enc -aes-256-cbc -pbkdf2 -md sha512 -iter 10000 -S 0 -iv 0 -in plaintext | xxd
```

```
openssl enc -aes-256-ctr -pbkdf2 -md sha512 -iter 10000 -S 0 -iv 0 -in plaintext | xxd
```

```
printf '7%.0s' {1..127} | openssl enc -aes-256-ecb -pbkdf2 -md sha512 -iter 10000 -iv 0 | xxd
```

```
printf '7%.0s' {1..127} | openssl enc -aes-256-cbc -pbkdf2 -md sha512 -iter 10000 -iv 0 | xxd
```

完全な非対称デジタル署名を試してみる

まず、公開鍵と秘密鍵を作成します。

```
openssl genrsa -out rsa.private 4096  
openssl rsa -in rsa.private -out rsa.public -pubout -outform PEM
```

次に、秘密鍵 `plaintext` を使用してファイルの署名を作成し、base64 でエンコードします。

```
openssl dgst -sha256 -sign rsa.private plaintext | base64 > plaintext.signature
```

これで、プロセスを逆にすることができます。まず、署名をファイルセーフなbase64ではなく、raw形式に変換する必要があります。

```
base64 -d plaintext.signature > raw
```

これで、公開鍵を使用してプレーンテキスト ファイルの署名を検証できます。

```
openssl dgst -sha256 -verify rsa.public -signature raw plaintext
```